Java Backend Interview Series

Sub-Chapter 1.7

Generics -- Deep Dive

Generic Classes / Wildcards / PECS / Type Erasure / Bridge Methods / Patterns

Complete Import Statements -- 30+ Q&A -- Type Theory -- Bridge Methods -- Real-World Patterns

1 Why Generics -- The Problem Before Java 5

Before Java 5, collections stored Object references. Every retrieval required an explicit cast, and ClassCastException lurked at runtime rather than being caught at compile time. A single misplaced add() could corrupt an entire collection silently. Generics moved these errors to compile time, eliminated casts, and made APIs self-documenting.

1.1 The Problem -- Raw Types and ClassCastException

```
import java.util.ArrayList;
import java.util.List;
// BEFORE Java 5 -- raw types, runtime danger
List prices = new ArrayList(); // raw type -- erased to Object
prices.add(9.99); // Double added
prices.add("FREE"); // String also accepted -- no compile error!

// ClassCastException at runtime, far from the bug:
for (Object o : prices) {
Double d = (Double) o; // ClassCastException on "FREE"
System.out.println(d * 1.1);
}
```

BEFORE generics: raw type causing runtime ClassCastException

1.2 The Solution -- Compile-Time Safety

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
// AFTER Java 5 -- parameterized type
List<Double> prices = new ArrayList<>();
prices.add(9.99);
// prices.add("FREE"); // COMPILE ERROR -- type mismatch

// No cast needed -- type is guaranteed
for (Double d : prices) {
System.out.println(d * 1.1);
}

// Self-documenting API -- no Javadoc needed to understand:
Map<String, List<Order>> customerOrders = new HashMap<>();
// vs Map rawMap = new HashMap(); -- what does this hold?
```

AFTER generics: compile-time safety, no casts, self-documenting

TIP: Generics are a compile-time-only feature. At runtime, List<String> and List<Integer> are the same class (java.util.List). This is called type erasure.

2 Generic Classes and Interfaces

A generic class or interface declares one or more type parameters in angle brackets. These parameters act as placeholders for actual types provided by callers. Type parameter naming convention: T=Type, E=Element, K=Key, V=Value, N=Number, R=Return type, S/U/V=2nd/3rd/4th types.

2.1 Generic Class -- Box<T> and Pair<A,B>

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
// Generic class with single type parameter
class Box<T> {
private T value;
public Box(T value) { this.value = value; }
public T getValue() { return value; }
public void setValue(T value) { this.value = value; }
@Override public String toString() { return "Box[" + value + "]"; }
}

// Generic class with two type parameters
class Pair<A, B> {
private final A first;
private final B second;
public Pair(A first, B second) { this.first = first; this.second = second; }
public A getFirst() { return first; }
public B getSecond() { return second; }
public static <A, B> Pair<A, B> of(A a, B b) { return new Pair<>(a, b); }
}

// Usage
Box<String> strBox = new Box<>("Hello"); // explicit type
Box<Integer> intBox = new Box<>(42); // autoboxing
Box<?> unknown = new Box<>("wildcard"); // unknown type
Pair<String, Integer> p = Pair.of("age", 30);

// Raw type -- avoid!
Box rawBox = new Box("raw"); // unchecked warning -- loses type safety
```

Generic Box and Pair with usage

2.2 Generic Interfaces and Repository Pattern

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
// Generic interface
interface Repository<T, ID> {
T findById(ID id);
List<T> findAll();
T save(T entity);
void delete(ID id);
}

// Implementing with specific types
class UserRepository implements Repository<User, Long> {
@Override public User findById(Long id) { return null; }
@Override public List<User> findAll() { return new ArrayList<>(); }
@Override public User save(User u) { return u; }
@Override public void delete(Long id) {}
}

// Implementing keeping type parameter open
class InMemoryRepository<T> implements Repository<T, Integer> {
private final Map<Integer, T> store = new HashMap<>();
@Override public T findById(Integer id) { return store.get(id); }
@Override public List<T> findAll() { return new ArrayList<>(store.values()); }
@Override public T save(T entity) {
store.put(store.size() + 1, entity); return entity;
}
@Override public void delete(Integer id) { store.remove(id); }
}
```

Generic Repository interface and implementations

3 Generic Methods -- Type Inference and Type Witness

A generic method declares its own type parameter before the return type. The compiler infers the type parameter from the arguments at the call site. When inference fails (ambiguous or insufficient context), an explicit type witness can be provided.

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.function.BiFunction;
import java.util.function.Predicate;
import java.util.stream.Collectors;
```

```
// Generic method -- type parameter declared before return type
public static <T> T identity(T value) { return value; }
public static <T> List<T> listOf(T a, T b) { return List.of(a, b); }

// Type inference at call site
String s = identity("hello"); // T inferred as String
Integer i = identity(42); // T inferred as Integer

// Explicit type witness when inference fails
List<String> empty = Collections.<String>emptyList();

// Generic methods in a utility class
class Utils {
// Swap two elements
public static <T> void swap(List<T> list, int i, int j) {
T temp = list.get(i);
list.set(i, list.get(j));
list.set(j, temp);
}

// Generic filter
public static <T> List<T> filter(List<T> list, Predicate<T> pred) {
return list.stream().filter(pred).collect(Collectors.toList());
}

// Generic zip
```

```
public static <A, B, R> List<R> zip(
List<A> as, List<B> bs, BiFunction<A, B, R> combiner) {
List<R> result = new ArrayList<>();
int size = Math.min(as.size(), bs.size());
for (int k = 0; k < size; k++)
result.add(combiner.apply(as.get(k), bs.get(k)));
return result;
}

// Bounded generic max
public static <T extends Comparable<T>> Optional<T> max(Collection<T> col) {
return col.stream().max(Comparator.naturalOrder());
}
}
```

Generic methods: swap, filter, zip, max

4 Bounded Type Parameters

Bounded type parameters restrict the set of types that can be used as type arguments. An upper bound (extends) allows the type to be used as the bound type or any subtype. Multiple bounds are combined with & -- the class bound must come first, followed by interface bounds.

4.1 Upper Bounded and Multiple Bounds

```
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
// T must implement Comparable<T>
public static <T extends Comparable<T>> T max(T a, T b) {
return a.compareTo(b) >= 0 ? a : b;
}

// T must extend Number -- can call Number methods on T
public static <T extends Number> double sum(List<T> list) {
return list.stream().mapToDouble(Number::doubleValue).sum();
}

// Multiple bounds: class bound FIRST, then interface bounds
public static <T extends Comparable<T> & Serializable & Cloneable>
void process(T value) {
System.out.println(value.compareTo(value)); // 0
}

// Bounded class definition
class SortedList<T extends Comparable<T>> {
private final List<T> list = new ArrayList<>();
public void add(T item) {
list.add(item);
Collections.sort(list); // valid -- T is Comparable
}
public T min() { return list.isEmpty() ? null : list.get(0); }
}
```

Upper bounded type parameters and multiple bounds

4.2 Recursive Generics and F-Bounded Quantification

```
import java.util.ArrayList;
```

```
// F-bounded: T refers to itself in its own bound
// Enables type-safe compareTo in subclasses
class Animal<T extends Animal<T>> implements Comparable<T> {
@Override public int compareTo(T other) { return 0; }
}
class Dog extends Animal<Dog> {}
// Dog.compareTo(Dog) is type-safe -- no Object cast

// The standard Enum uses this:
// class Enum<E extends Enum<E>> -- every enum has this bound

// Builder pattern with self-referential generic
abstract class Builder<T, B extends Builder<T, B>> {
@SuppressWarnings("unchecked")
protected B self() { return (B) this; }
public B withName(String name) { /* set name */ return self(); }
public abstract T build();
}
class PersonBuilder extends Builder<Person, PersonBuilder> {
@Override public Person build() { return new Person(); }
}
// Fluent call returns PersonBuilder (not Builder), no cast:
// Person p = new PersonBuilder().withName("Alice").build();
```

F-bounded quantification and generic Builder pattern

5 Wildcards Deep Dive -- PECS

PECS: Producer Extends, Consumer Super. If the collection PRODUCES values you READ from it, use <? extends T>. If it CONSUMES values you WRITE to it, use <? super T>. If you both read and write, use the exact type T. Unbounded <?> means you only use the Object API on elements.

5.1 Unbounded Wildcard <?>

```
import java.util.List;
// <?> -- do not care about element type; only Object API available
void printAll(List<?> list) {
for (Object item : list) System.out.println(item);
// list.add("x"); // COMPILE ERROR -- type unknown
// list.add(null); // OK -- null is always valid
}

// List<?> accepts any List<T>:
List<String> strings = List.of("a","b");
List<Integer> ints = List.of(1, 2);
List<?> anyList = strings; // OK
anyList = ints; // OK

// List<Object> does NOT accept List<String>:
// List<Object> objects = strings; // COMPILE ERROR -- generics are invariant
```

Unbounded wildcard -- printAll example

5.2 Upper Bounded <? extends T> -- Producer Extends

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
// <? extends Number> -- holds some subtype of Number
double sumNumbers(List<? extends Number> src) {
double total = 0;
for (Number n : src) total += n.doubleValue(); // reading as Number: OK
return total;
}
sumNumbers(new ArrayList<Integer>()); // OK
sumNumbers(new ArrayList<Double>()); // OK

// Cannot add to <? extends Number> -- compiler does not know exact type
List<? extends Number> nums = new ArrayList<Integer>();
```

```
// nums.add(3.14); // COMPILE ERROR -- might be List<Integer>!

nums.add(null); // only null is always valid

// Collections.copy uses PECS:

// static <T> void copy(List<? super T> dest, List<? extends T> src)

List<Integer> src = List.of(1, 2, 3);

List<Number> dest = new ArrayList<>(List.of(0, 0, 0));

Collections.copy(dest, src); // dest=consumer, src=producer
```

Upper bounded wildcard -- Producer Extends

5.3 Lower Bounded <? super T> -- Consumer Super

```
import java.util.ArrayList;

import java.util.List;

// <? super Integer> -- holds Integer or any supertype (Number, Object)

void addIntegers(List<? super Integer> dest, int count) {

for (int i = 0; i < count; i++) dest.add(i); // writing Integer: OK

}

addIntegers(new ArrayList<Integer>(), 5); // OK

addIntegers(new ArrayList<Number>(), 5); // OK

addIntegers(new ArrayList<Object>(), 5); // OK

// Cannot read typed values from <? super Integer>

List<? super Integer> dest2 = new ArrayList<Number>();

// Integer x = dest2.get(0); // COMPILE ERROR -- might be Number or Object

Object o = dest2.get(0); // only Object available when reading
```

Lower bounded wildcard -- Consumer Super

Wildcard	Read as	Can write	Use case
<?>	Object	null only	Print/count; no type-specific ops needed
<? extends T>	T (upper bound)	null only	Reading/processing; Producer pattern
<? super T>	Object	T and subtypes	Writing/collecting; Consumer pattern
<T> (exact)	T	T	Both reading and writing needed

6 Type Erasure -- What the JVM Actually Sees

Type erasure: Java generics exist at compile time only. The compiler enforces type rules then erases all generic information. At runtime, List<String> and List<Integer> are the same class: java.util.List. Erasure was chosen for backward compatibility with pre-Java-5 bytecode.

6.1 What Gets Erased

```
import java.util.List;
// Source code (before erasure)
class Container<T extends Comparable<T>> {
private T value;
public T get() { return value; }
public void set(T v) { this.value = v; }
public boolean bigger(T other) { return value.compareTo(other) > 0; }
}

// After erasure (what JVM sees -- approximately)
class Container {
private Comparable value; // T -> first bound (Comparable)
public Comparable get() { return value; }
public void set(Comparable v) { this.value = v; }
public boolean bigger(Comparable other) {
return value.compareTo(other) > 0;
}
}
// Unbounded T erases to Object
// Bounded T erases to its first bound
// Multiple bounds <T extends A & B>: T erases to A; casts to B are inserted

// Erasure consequences:
// 1. No instanceof with parameterized type:
// if (obj instanceof List<String>) {} // COMPILE ERROR
// if (obj instanceof List<?>) {} // OK
// 2. Cannot create: new T(); T[] arr = new T[10];
// 3. Cannot catch: catch (SomeException<T> e) {}
// 4. Cannot overload: void f(List<String>) + void f(List<Integer>) -- ERROR
// 5. Static members cannot use class type parameter
```

Type erasure -- before and after; consequences

6.2 Heap Pollution and @SuppressWarnings

```
import java.util.ArrayList;
import java.util.List;
// Heap pollution: variable of parameterized type holds wrong type
List<String> strings = new ArrayList<>();
List rawList = strings; // raw type assignment -- unchecked
rawList.add(42); // adds Integer to List<String> -- NO error!
String s = strings.get(0); // ClassCastException at RUNTIME!

// @SuppressWarnings("unchecked") -- use ONLY when you KNOW it is safe
@SuppressWarnings("unchecked")
static <T> T cast(Object obj) {
return (T) obj; // unchecked cast -- safe when caller guarantees type
}

// Type token to check at runtime despite erasure:
class TypedList<T> {
private final Class<T> type;
private final List<T> list = new ArrayList<>();
public TypedList(Class<T> type) { this.type = type; }
public void add(Object o) {
if (!type.isInstance(o))
throw new ClassCastException("Expected " + type.getName());
list.add(type.cast(o)); // safe checked cast
}
}
```

Heap pollution, @SuppressWarnings, and type token workaround

> WARNING: @SuppressWarnings("unchecked") is a PROMISE to the compiler that you have verified the cast is safe. Always add a comment explaining why. Never use it to silence warnings you do not understand.

7 Generic Arrays -- Why new T[] Fails and Workarounds

Generic array creation (new T[]) is forbidden because arrays carry runtime type information (reifiable types), while generic type parameters are erased. Creating a T[] would circumvent the array store check, enabling heap pollution that would only surface as a ClassCastException far from the actual bug.

```
import java.util.ArrayList;
import java.util.List;
import java.lang.reflect.Array;
// COMPILE ERROR: Cannot create generic array
// T[] arr = new T[10];

// Workaround 1: Unchecked cast with suppression (common in JDK itself)
@SuppressWarnings("unchecked")
T[] createArray(int size) {
return (T[]) new Object[size]; // safe if T is not exposed as T[] to caller
}

// Workaround 2: Class<T> token for type-safe array creation
T[] createArray(Class<T> type, int size) {
return (T[]) java.lang.reflect.Array.newInstance(type, size);
}

// Workaround 3 (preferred): Use List<T> -- avoids the issue entirely
List<T> createList(int size) {
return new ArrayList<>(size);
}

// Generic Stack using List (no array issues)
class GenericStack<E> {
private final List<E> elements = new ArrayList<>();
public void push(E e) { elements.add(e); }
public E pop() { return elements.remove(elements.size()-1); }
public E peek() { return elements.get(elements.size()-1); }
public boolean isEmpty() { return elements.isEmpty(); }
public int size() { return elements.size(); }
}
```

Generic array workarounds -- unchecked cast, Class token, List

WARNING: Prefer List<T> over T[] in generic code. Lists avoid heap pollution risks from generic arrays and integrate naturally with the Collections API and streams.

8 Recursive Generics and @SafeVarargs

8.1 Generic Builder with Self-Type

```
import java.util.LinkedHashMap;
import java.util.Map;
// Fluent generic builder -- chaining returns correct subtype
class HttpRequest {
final String method, url;
final Map<String, String> headers;

protected HttpRequest(Builder<?> b) {
this.method = b.method;
this.url = b.url;
this.headers = Map.copyOf(b.headers);
}

static class Builder<B extends Builder<B>> {
String method = "GET", url;
Map<String, String> headers = new LinkedHashMap<>();

@SuppressWarnings("unchecked")
public B method(String m) { this.method = m; return (B) this; }
@SuppressWarnings("unchecked")
public B url(String u) { this.url = u; return (B) this; }
@SuppressWarnings("unchecked")
public B header(String k, String v) { headers.put(k,v); return (B) this; }
public HttpRequest build() { return new HttpRequest(this); }
}
}

// Subclass extends builder without needing casts:
class AuthenticatedRequest extends HttpRequest {
final String token;
protected AuthenticatedRequest(Builder b) {
super(b); this.token = b.token;
}
static class Builder extends HttpRequest.Builder<Builder> {
String token;
```

```
public Builder token(String t) { this.token = t; return this; }

@Override public AuthenticatedRequest build() {

return new AuthenticatedRequest(this);

}

}

}

AuthenticatedRequest req = new AuthenticatedRequest.Builder()

.method("POST").url("/api/data").token("Bearer xyz").build();
```

Generic Builder -- self-referential type for fluent chaining

8.2 @SafeVarargs -- Heap Pollution in Varargs

```
import java.util.ArrayList;

import java.util.List;

// Generic varargs creates an array T... -> T[] -- triggers heap pollution warning

// @SafeVarargs suppresses the warning IF the method is truly safe

// SAFE -- only reads from the varargs array, does not store into it

@SafeVarargs

public static <T> List<T> listOf(T... elements) {

List<T> result = new ArrayList<>();

for (T e : elements) result.add(e); // read-only -- safe

return result;

}

// UNSAFE -- stores a different parameterized type into the array

// @SafeVarargs would be a lie here:

static void unsafe(List<String>... lists) {

Object[] arr = lists; // legal -- array covariance

arr[0] = List.of(42); // heap pollution! List<Integer> in List<String>[]

String s = lists[0].get(0); // ClassCastException at RUNTIME

}

// @SafeVarargs requirements:

// - Method must be final, static, or private (Java 9+: constructors too)

// - Promise: method does NOT store into the varargs array

// - Promise: method does NOT expose reference to the varargs array
```

@SafeVarargs -- safe vs unsafe varargs

9 Generics in Practice -- Real-World Patterns

9.1 Type-Safe Heterogeneous Container

```
import java.util.HashMap;
import java.util.Map;
import java.util.Objects;
// Problem: Map<Class<?>, Object> loses type safety
// Solution: type token -- Class<T> carries runtime type despite erasure
class TypeSafeContainer {
private final Map<Class<?>, Object> map = new HashMap<>();

public <T> void put(Class<T> type, T value) {
map.put(Objects.requireNonNull(type), Objects.requireNonNull(value));
}

public <T> T get(Class<T> type) {
return type.cast(map.get(type)); // safe -- throws if wrong type
}
}

TypeSafeContainer c = new TypeSafeContainer();
c.put(String.class, "hello");
c.put(Integer.class, 42);
String s = c.get(String.class); // "hello" -- type safe
Integer i = c.get(Integer.class); // 42 -- type safe

// Limitation: cannot distinguish List<String> vs List<Integer>
// -- same Class token (List.class)
// Solution: Guava TypeToken or Jackson TypeReference for parameterized types
```

TypeSafeContainer -- type token pattern

9.2 Generic Result Type -- Success / Failure Monad

```
import java.util.function.Function;
// Type-safe Result monad -- avoids checked exceptions for expected failures
sealed interface Result<T> permits Result.Success, Result.Failure {
record Success<T>(T value) implements Result<T> {}
record Failure<T>(String message, Throwable cause) implements Result<T> {}

static <T> Result<T> success(T value) { return new Success<>(value); }
```

```
static <T> Result<T> failure(String msg, Throwable t) {
return new Failure<>(msg, t);
}

default <R> Result<R> map(Function<T, R> fn) {
return switch (this) {
case Success<T> s -> Result.success(fn.apply(s.value()));
case Failure<T> f -> Result.failure(f.message(), f.cause());
};
}

default T getOrElse(T defaultValue) {
return switch (this) {
case Success<T> s -> s.value();
case Failure<T> f -> defaultValue;
};
}
}

// Usage: chain fallible operations without try-catch clutter
// Result<User> result = userService.findUser(id);
// String name = result.map(User::getName).getOrElse("Unknown");
```

Generic Result sealed type -- Success/Failure monad

! Common Pitfalls and Anti-Patterns

Anti-Pattern	Why It Fails	Correct Approach
new T()	T erased to Object; constructor not guaranteed	Pass Class<T> token or Supplier<T> factory
T[] arr = new T[n]	Generic array -- heap pollution risk	Use (T[])new Object[n] with @SuppressWarnings or List<T>
catch (SomeException<T> e)	Catch clause requires reifiable type	Catch raw SomeException; check type manually
instanceof List<String>	Parameterized types not reifiable	Use instanceof List<?> then cast with check
static T field	T is per-instance; static is per-class	Use static generic method or non-generic static field
Overload by generic type alone	List<String> and List<Integer> same erasure	Use different method names or add Class<T> param
Raw type usage	Bypasses all type checking; heap pollution	Always specify type parameter; use <?> for unknown
Ignore unchecked warnings	May indicate real heap pollution bug	Fix or verify safety, then @SuppressWarnings with comment

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Supplier;
// WRONG: cannot overload by generic type -- same erasure
// void process(List<String> list) {} // COMPILE ERROR
// void process(List<Integer> list) {}

// FIX 1: different method names
void processStrings(List<String> list) { /* ... */ }
void processIntegers(List<Integer> list) { /* ... */ }

// FIX 2: add Class<T> parameter
<T> void process(List<T> list, Class<T> type) { /* ... */ }

// WRONG: cannot use new T() -- type erased at runtime
// <T> T create() { return new T(); } // COMPILE ERROR

// FIX: pass Supplier<T> factory (preferred)
<T> T create(Supplier<T> factory) { return factory.get(); }
// Usage: T obj = create(ArrayList::new);

// FIX: pass Class<T> and use reflection (not preferred)
```

```
<T> T create(Class<T> type) throws Exception {

return type.getDeclaredConstructor().newInstance();

}
```

Common generics pitfalls and fixes

Q Interview Q&A -- 30 Questions

Q1. What is the purpose of generics in Java?

Generics provide compile-time type safety, eliminate explicit casts, and enable code reuse across different types. Instead of List of Object requiring casts, List<String> guarantees all elements are Strings, with ClassCastException prevented at compile time. They also make APIs self-documenting.

Q2. What is a raw type and why should you avoid it?

A raw type is a generic class/interface used without type arguments: List instead of List<String>. It defeats all compile-time type checking. A raw List accepts any object; retrieving causes ClassCastException at an unexpected place. Only use raw types when interfacing with legacy pre-generics APIs.

Q3. What is the diamond operator <> ?

Diamond operator (Java 7+) lets the compiler infer type parameters on the right side: Map<String,List<Integer>> map = new HashMap<>();. Before Java 7: new HashMap<String,List<Integer>>(). Diamond is syntactic sugar -- same bytecode, improved readability for complex nested generics.

Q4. What are type parameter naming conventions?

Standard: T=general Type, E=Element (collections), K=Key, V=Value (maps), N=Number, R=Return type, S/U/V=2nd/3rd/4th types. These are conventions not rules. Single uppercase letters stand out from class names. For domain-specific generics, descriptive names like <ENTITY> can be clearer.

Q5. Can a generic class extend a non-generic class?

Yes to both directions. Generic extends non-generic: class NumberBox<T> extends Object {}. Non-generic extends generic: class StringList extends ArrayList<String> {} -- fixes the type argument. Also: class RawList extends ArrayList {} -- raw extends (not recommended).

Q6. What is the difference between a bounded type parameter and a bounded wildcard?

Bounded type parameter in declaration: <T extends Number> -- T is a placeholder reusable throughout the declaration. Bounded wildcard in usage: List<? extends Number> -- means some unknown specific subtype. Key: with <T> you can relate multiple uses to the same type. With wildcard, each occurrence is independent.

Q7. How do you call a generic method with an explicit type?

Use the type witness before the method name: Collections.<String>emptyList(). Usually unnecessary with Java 8+ improved target-type inference, but needed when inference fails -- e.g., when the return type is used in an overloaded call or assigned to Object.

Q8. Can you overload a method that differs only in generic type argument?

No. After erasure, List<String> and List<Integer> both become List. Two methods void process(List<String>) and void process(List<Integer>) have identical erasures -- compile error. Fix: use different method names or add a Class<T> parameter to disambiguate.

Q9. Explain PECS with a concrete example.

Producer Extends, Consumer Super. Collections.copy(List<? super T> dest, List<? extends T> src): src PRODUCES elements (you read from it) so extends. dest CONSUMES elements (you write to it) so super. Another: void pushAll(Iterable<? extends T> src) reads from src (producer=extends). void popAll(Collection<? super T> dst) writes to dst (consumer=super).

Q10. What is the difference between List<?> and List<Object>?

List<Object> only accepts List<Object> itself -- not List<String>, due to invariance. You can add any Object to List<Object>. List<?> accepts ANY List<T> for any T. You cannot add anything to List<?> (except null) because the exact type is unknown. List<?> is useful when you only need structural operations: size(), clear(), isEmpty().

Q11. Why can't you add to a List<? extends Number>?

The compiler does not know the exact type. List<? extends Number> could be List<Integer>, List<Double>, or List<Number>. If you could add a Double to what is actually a List<Integer>, you would corrupt the list. The compiler prevents all adds (except null) conservatively.

Q12. Why can't you read typed values from List<? super Integer>?

List<? super Integer> could be List<Integer>, List<Number>, or List<Object>. get() can only return Object because the compiler does not know which. You can add Integer (any List<? super Integer> can hold it), but reading requires a cast: Object o = list.get(0); Integer i = (Integer) o;

Q13. What is type erasure and why does Java use it?

Generic type information is erased by the compiler and replaced with bounds (or Object) at runtime. List<String> and List<Integer> are both just List at runtime. Java uses erasure for backward compatibility -- pre-Java-5 bytecode could use Java 5+ generic classes without recompilation.

Q14. What are the practical limitations of type erasure?

1) No instanceof List<String> -- use instanceof List<?>. 2) No new T() -- pass Supplier<T> or Class<T>. 3) No T[] -- use List<T> or (T[])new Object[n]. 4) No catch (GenEx<T> e). 5) No overloading by generic type. 6) No T.class. 7) Reflection only preserves generic info for fields/method signatures, not local variables.

Q15. What is heap pollution and when does it occur?

Heap pollution: a variable of parameterized type refers to an object that is not of that type. Occurs when mixing raw types with generic types, or with unchecked casts. List<String> strings = new ArrayList<>(); ((List)strings).add(42); ClassCastException occurs only when you read the element as String -- potentially far from the bug.

Q16. [ADV] Why are Java generics invariant while arrays are covariant?

Arrays are covariant: String[] IS-A Object[]. Generics are invariant: List<String> IS NOT List<Object>. Array covariance was a Java 1.0 design mistake: Object[] arr = new String[5]; arr[0] = 42; -- compiles but throws ArrayStoreException (runtime check per element). Generics chose invariance + wildcards for compile-time safety with no runtime overhead.

Q17. [ADV] What exactly does type erasure do for <T extends Comparable<T> & Serializable>?

T erases to its first bound: Comparable. Where Serializable methods are referenced, the compiler inserts a checkcast to Serializable at usage sites. Only the first bound appears in the erased signature. This is why the class bound must come first in <T extends Clazz & Interface1 & Interface2>.

Q18. [ADV] Is PECS enforced by the JVM or only the compiler?

Purely compile-time. The JVM has no knowledge of wildcards. At bytecode level, List<? extends Number> is just List. The compiler tracks wildcards in its symbol table and rejects illegal method calls. You can bypass PECS at runtime via raw types or unchecked casts -- the JVM will not stop you, but ClassCastException will occur later.

Q19. [ADV] What is wildcard capture and why does it require a helper method?

When the compiler processes a wildcard-typed expression, it captures the wildcard into a fresh type variable (CAP#1). Two uses of the same List<?> variable get different captures: list.set(0, list.get(0)) fails because list.get(0)

returns CAP#1 and list.set(0,...) expects CAP#2. A helper <T> void helper(List<T> l) { l.set(0, l.get(0)); } unifies both under one T.

Q20. [ADV] Explain the recursive generic pattern class Enum<E extends Enum<E>>.

F-bounded quantification: every enum implicitly has this bound. It enables Enum.compareTo(E other) to be type-safe -- you can only compare an enum with members of the same enum type. Without it, compareTo would accept any Enum. The pattern generalizes to fluent APIs: Builder<B extends Builder<B>> lets methods return B (the actual subtype) not Base.

Q21. [ADV] When is @SafeVarargs appropriate?

@SafeVarargs on a final/static/private generic varargs method suppresses heap pollution warnings. Safe when: (1) method does NOT store into the varargs array; (2) method does NOT expose a reference to the array. Arrays.asList(T... a) is @SafeVarargs -- read-only. Not safe if you assign elements of a different parameterized type into the array.

Q22. [ADV] What is a bridge method and when does the compiler generate one?

Compiler generates a synthetic bridge method for erasure + polymorphism. class SP implements Processor<String> -- the compiler generates process(Object) delegating to process(String), so the JVM can call Processor.process(Object) polymorphically. Also generated for covariant return types. View with: javap -p -verbose ClassName -- flagged ACC_SYNTHETIC + ACC_BRIDGE.

Q23. [ADV] How do bridge methods interact with reflection?

getDeclaredMethods() returns ALL methods including bridges. Filter with method.isBridge() or method.isSynthetic(). Spring AOP, Jackson, and other reflection-heavy frameworks must filter synthetics to avoid invoking the wrong method or getting confusing method-not-found errors.

Q24. [ADV] How does Jackson's TypeReference work under the hood?

TypeReference uses the super-type token pattern. new TypeReference<List<User>>(){} creates an anonymous subclass. Its generic supertype is TypeReference<List<User>> -- a ParameterizedType. TypeReference constructor calls getClass().getGenericSuperclass() to capture it. ObjectMapper uses the resulting Type to determine the element type (User.class) and build a deserializer.

Q25. [ADV] Why can't you do new T() and what are the design alternatives?

new T() is forbidden: after erasure T is Object; T might lack a no-arg constructor; T might be an interface or abstract class. Alternatives: (1) Supplier<T> factory: Supplier<T> factory; T obj = factory.get(); -- preferred, no reflection. (2) Class<T> token: type.getDeclaredConstructor().newInstance(); (3) Prototype + clone.

Q26. [ADV] What is the difference between Class.cast() and an explicit (T) cast?

Class.cast(obj) is a checked cast: calls isInstance() first, throws ClassCastException immediately with a clear message if types mismatch. (T) obj is unchecked -- at bytecode level it is either a checkcast instruction (if T is known) or completely removed (if T erased to Object). Class.cast() fails at the cast site; (T) obj may fail much later when the value is used.

Q27. [ADV] What is the get-and-put principle?

Get-and-put (from Java Generics and Collections book): Use <? extends T> when you only GET values out (producer). Use <? super T> when you only PUT values in (consumer). Use exact type when you do both. This is PECS described from the structure's perspective rather than the data's perspective.

Q28. [ADV] What is co-variance, contra-variance, and invariance in generics?

Invariance: List<Dog> IS-NOT-A List<Animal>. Covariance: List<? extends Animal> -- read-only, analog of Kotlin out T. Contravariance: List<? super Dog> -- write-only for Dog, analog of Kotlin in T. Java uses use-site variance (wildcards declared at each usage). Kotlin/Scala support declaration-site variance (out/in declared on the class itself).

Q29. [ADV] How does Guava TypeToken differ from TypeReference?

TypeReference (Jackson/custom): captures one ParameterizedType via anonymous subclass. Simple for deserializing into a specific generic type. Guava TypeToken: a full type system over Java's Type hierarchy; supports type operations: resolveType(T), getSubtype(), getSupertype(), isSubtypeOf(). Use TypeReference for simple serialization; Guava TypeToken for framework-level generic type reasoning.

Q30. [ADV] What is Kotlin's reified type parameter and how does it solve erasure?

Kotlin inline functions with reified type parameters embed actual type info at call sites. inline fun <reified T> isType(obj: Any) = obj is T -- compiles to a real instanceof check because the function is inlined (separate bytecode per call site). Java cannot do this -- non-inline methods share one bytecode for all T. Java workaround: always pass Class<T> for operations needing runtime type.

S Summary -- Key Takeaways

Topic	Key Points
Why Generics	Compile-time safety; no casts; self-documenting API; ClassCastException at compile not runtime
Generic class/interface	class Box<T>; interface Repo<T,ID>; diamond <>; raw types bypass type safety
Generic methods	<T> before return type; type inference at call site; type witness Collections.<String>emptyList()
Bounded params	<T extends Comparable<T>>; multiple bounds: class first then interfaces; erasure to first bound
Wildcards	<?>=Object-only; <? extends T>=read/producer; <? super T>=write/consumer
PECS	Producer Extends, Consumer Super -- the single most important wildcard rule
Type erasure	Compile-time only; List<String> == List at runtime; no instanceof, new T[], catch
Heap pollution	Raw+generic mixing; @SuppressWarnings("unchecked") only when verified safe
Generic arrays	new T[] forbidden; use List<T> or (T[])new Object[n] or Array.newInstance
Recursive generics	<B extends Builder<B>>; F-bounded; @SafeVarargs for read-only varargs
Bridge methods	Compiler-generated for erasure+polymorphism; isBridge()=true; filter in reflection
Type token	Class<T> for runtime type; anonymous subclass for ParameterizedType (TypeReference)

Next: Sub-Chapter 1.8 -- Exception Handling